Bitácora de Ghidra: CODEBREAKERS

El misterio del código oculto: paso a paso del análisis

Brayan Loaiza Leal, *Student, ECI-ISIS*

***Abstract*—Este documento registra, en el orden en que se hizo, cómo se analizó en Ghidra el programa del reto CODEBREAKERS hasta encontrar la clave y obtener la FLAG, sin acceso al código fuente. Se parte de una ejecución con una clave falsa, se importa el binario en Ghidra, se localizan las cadenas del programa y se sigue la referencia al mensaje de acceso hasta `main`. Allí se identifica un descifrado AES-256-CBC cuya clave e IV están en texto plano dentro del ejecutable; se reproduce el descifrado con OpenSSL y se comprueba la clave obtenida ejecutando el programa.**

***Keywords*—Ghidra, ingeniería inversa, CTF, AES-256-CBC, análisis estático.**

I. Datos generales

- **Fecha:** 2026-11-11.
- **Herramienta:** Ghidra 12.1.4 (release oficial de la NSA) sobre JDK 26, en Windows 11. Proyecto `ECI_CODEBREAKERS`.
- **Apoyo:** WSL2 Ubuntu 24.04 para ejecutar el programa y OpenSSL para reproducir el descifrado.
- **Binario:** `reto_codebreakers_linux_x64`, ELF 64-bit PIE, x86-64, dinámico, *not stripped*.
- **SHA-256:**

```
8676a8798847da67b0449092e4341eaaf21a995a37d7
aea916485b1503dc717f
```

- **Resultado:** la clave `BREAKER#2026` produce `ACCESO CONCEDIDO` y `FLAG{CODEBREAKERS_ECI}`.

A. Flujo general

```
Ejecutar (clave falsa)
 -> Importar en Ghidra -> Auto Analyze
 -> Search for Strings
 -> "ACCESO CONCEDIDO" -> References
 -> main -> Decompiler:
    strcmp(entrada, descifrado)
 -> que es "descifrado"?
    sbox / rsbox / add_round_key
    => AES-256-CBC
 -> clave e IV en 0x102300 y 0x102320,
    cifrado en aes_ct
 -> descifrar con OpenSSL
 -> BREAKER#2026 -> ejecutar -> FLAG
```

II. Paso 0. Primer intento sin análisis

Antes de abrir Ghidra se ejecutó el programa en WSL con una palabra cualquiera para conocer su comportamiento:

```
$ chmod +x reto_codebreakers_linux_x64
$ ./reto_codebreakers_linux_x64
=== CODEBREAKERS: BOVEDA DIGITAL ECI ===
Desafio: descubre la clave sin recibir
el codigo fuente.
Introduce la clave: prueba
ACCESO DENEGADO
```

Observación: el programa pide la clave por la entrada estándar (no como argumento) y responde `ACCESO DENEGADO`. El objetivo es encontrar qué texto hace que responda lo contrario.

III. Paso 1. Crear el proyecto e importar

TABLE I: Creación del proyecto e importación del binario.

#	Acción	Resultado
1.1	Abrir `ghidraRun.bat` desde la carpeta de Ghidra	Ghidra 12.1.4 arranca con JDK 26.
1.2	File → New Project → Non-Shared Project, nombre `ECI_CODEBREAKERS`	Proyecto local, sin servidor.
1.3	File → Import File… → `reto_codebreakers_linux_x64`	Formato ELF, lenguaje `x86:LE:64:default`, compilador gcc. Base de imagen `0x100000`.
1.4	Doble clic en el programa → Analyze now? → Yes, opciones por defecto	Análisis automático de funciones, cadenas, referencias y decompilación.

Observación: el binario no está *stripped*. En Symbol Tree → Functions aparecen nombres útiles: `main` (`0x1010c0`) y `add_round_key` (`0x101ba0`), y en los datos `sbox`, `rsbox` y `aes_ct`.

IV. Paso 2. Buscar cadenas

TABLE II: Búsqueda de cadenas.

#	Acción	Resultado
2.1	Search → For Strings…, opciones por defecto	Lista de todas las cadenas del binario.
2.2	Revisar los mensajes del programa	`ACCESO CONCEDIDO` (`0x1020a2`), `FLAG{CODEBREAKERS_ECI}` (`0x1020b3`), `ACCESO DENEGADO` (`0x1020ca`).
2.3	Buscar algo con forma de clave	Una cadena de 48 caracteres en `0x102300` (ver abajo).

```
CodeBreakersECI_AES256_KEY_2026!
ECI_VECTOR_2026!
```

Pista encontrada: la cadena larga no es la clave del reto: anuncia `AES256_KEY` y `VECTOR`. Esto indica que la clave del reto está cifrada con AES-256 y que esa cadena contiene

Fundamentos de Seguridad Informática (FDSI), Escuela Colombiana de Ingeniería Julio Garavito, 2026. Documento para el monitor: contiene la clave del reto; no publicarlo antes de que los compañeros lo intenten.

la clave AES y el vector de inicialización (IV). Se verificó con `strings`:

```
$ strings -n 6 reto_codebreakers_linux_x64 \
    | grep -E 'ACCESO|FLAG|CodeBreakers'
ACCESO CONCEDIDO
FLAG{CODEBREAKERS_ECI}
ACCESO DENEGADO
CodeBreakersECI_AES256_KEY_2026!ECI_VECTOR_2026!
```

V. Paso 3. Seguir la referencia a ACCESO CONCEDIDO

TABLE III: Seguimiento de la referencia hasta `main`.

#	Acción	Resultado
3.1	Doble clic en `ACCESO CONCEDIDO` en la lista de cadenas	Se abre el Listing en `0x1020a2`.
3.2	Clic derecho → References → Show References to Address	Una sola referencia, en `0x101a5b`, dentro de `main`.
3.3	Doble clic en la referencia y abrir la ventana Decompiler	Se ve el `if` que decide entre los dos mensajes.

Final de `main` en el Decompiler:

```
local_1c8 = _DAT_00102320 ^ local_1c8;
 // XOR con el IV (modo CBC)
uStack_1c0 = _UNK_00102328 ^ uStack_1c0;
bStack_1c9 = (byte)(uStack_1c0 >> 0x38);
 // ultimo byte = relleno
if ((byte)(bStack_1c9 - 1) < 0x10) {
  *(undefined1 *)((long)&local_1c8 +
     (long)(int)(0x10 - (uint)bStack_1c9)) = 0;
}
iVar21 = strcmp((char *)local_1a8,
                (char *)&local_1c8);
if (iVar21 == 0) {
  puts("ACCESO CONCEDIDO");
  puts("FLAG{CODEBREAKERS_ECI}");
}
else {
  puts("ACCESO DENEGADO");
}
```

Conclusión: la entrada del usuario (`local_1a8`, leída con `fgets`) se compara con `strcmp` contra un búfer de 16 bytes (`local_1c8`). Ese búfer no está en claro en el binario: `main` lo calcula antes de comparar. Hay que entender cómo.

VI. Paso 4. Identificar el algoritmo

Se leyó `main` desde el inicio en el Decompiler, renombrando lo que se iba entendiendo.

TABLE IV: Lectura de `main` en el Decompiler.

#	Qué se observó en Ghidra	Interpretación
4.1	`local_128 = _DAT_00102300 ... uStack_110 = _UNK_00102318`: se copian 32 bytes desde `0x102300`	Los primeros 32 caracteres de la cadena larga se cargan como clave.
4.2	Bucle `do ... while (uVar23 != 0x3c)` que usa `sbox[...]` y `1 « (...)` cada 8 palabras	Expansión de clave de AES-256: 60 palabras (`0x3c`) y la constante de ronda.
4.3	`puVar19 = &aes_ct;` copia 16 bytes de `aes_ct` (`0x1020e0`)	Es el texto cifrado: un solo bloque AES de 16 bytes.
4.4	`puVar19 = rsbox;` y llamadas a `add_round_key(local_1b8, &local_128, ...)`	`rsbox` es la S-box inversa de AES: el programa descifra, no cifra.
4.5	`local_1c8 = _DAT_00102320 ^ local_1c8` (16 bytes desde `0x102320`)	XOR con el IV tras descifrar el bloque: modo CBC. El IV es `ECI_VECTOR_2026!`.
4.6	Se toma el último byte y se pone un 0 en la posición `0x10 - relleno`	Quita el relleno PKCS#7 y termina la cadena para `strcmp`.

TABLE V: Datos identificados en el binario.

Dato	Dirección	Valor
Clave AES-256 (32 B)	`0x102300`	`CodeBreakersECI_AES256_KEY_2026!`
IV (16 B)	`0x102320`	`ECI_VECTOR_2026!`
`aes_ct` (16 B)	`0x1020e0`	`d67f5789f451afebf2b6e566f87fb979`
`rsbox` (256 B)	`0x102100`	S-box inversa de AES
`sbox` (256 B)	`0x102200`	S-box de AES, usada en la expansión de clave

VII. Paso 5. Descifrar la clave

Con la clave, el IV y el texto cifrado identificados en Ghidra, se reprodujo en WSL el mismo descifrado que hace `main`:

```
KEY=436f6465427265616b6572734543495f
    4145533235365f4b45595f3230323621
IV=4543495f564543544f525f3230323621
CT=d67f5789f451afebf2b6e566f87fb979
$ printf '%s' "$CT" | xxd -r -p | \
  openssl enc -d -aes-256-cbc -K $KEY -iv $IV
BREAKER#2026
```

OpenSSL retira el relleno PKCS#7 igual que el programa. El resultado es la clave: `BREAKER#2026`.

VIII. Paso 6. Comprobar ejecutando el programa

```
$ ./reto_codebreakers_linux_x64
=== CODEBREAKERS: BOVEDA DIGITAL ECI ===
Desafio: descubre la clave sin recibir
el codigo fuente.
Introduce la clave: BREAKER#2026
ACCESO CONCEDIDO
FLAG{CODEBREAKERS_ECI}
```

TABLE VI: Comparación de las dos pruebas.

Prueba	Entrada	Salida
Clave falsa	`prueba`	`ACCESO DENEGADO`
Clave descubierta	`BREAKER#2026`	`ACCESO CONCEDIDO` y `FLAG{CODEBREAKERS_ECI}`

IX. Evidencia para el monitor

TABLE VII: Respuestas a la guía del reto.

Pregunta de la guía	Respuesta
¿Cuál es la clave?	`BREAKER#2026`
¿Qué palabra o función llevó a la conclusión?	La cadena `CodeBreakersECI_AES256_KEY_2026!ECI_VECTOR_2026!`, la referencia de `ACCESO CONCEDIDO` hacia `main`, el `strcmp` de `main` y los símbolos `rsbox`, `add_round_key` y `aes_ct`, que identifican un descifrado AES-256-CBC.
¿Qué mensaje produce la clave correcta?	`ACCESO CONCEDIDO` seguido de `FLAG{CODEBREAKERS_ECI}`

X. Atajos de Ghidra usados

TABLE VIII: Atajos y menús empleados.

Atajo o menú	Acción
Search → For Strings…	Listar las cadenas del binario
Clic derecho → References → Show References to Address	Ver desde dónde se usa una cadena o un dato
Doble clic	Seguir una referencia, una llamada o un dato
`L`	Renombrar variable, función o etiqueta
`;`	Agregar comentario
`G`	Ir a una dirección, por ejemplo `0x102300`

XI. Reflexión

1) **La clave viaja con el programa.** La clave AES y el IV están dentro del mismo ejecutable, en texto plano. Cifrar la contraseña solo añade un paso: cualquiera que lea el binario puede descifrarla.
2) **La FLAG no necesita la clave.** `FLAG{CODEBREAKERS_ECI}` está en texto plano y aparece con un simple `strings`.
3) **Los símbolos ayudan al analista.** Al no estar *stripped*, nombres como `rsbox`, `add_round_key` y `aes_ct` delatan el algoritmo de inmediato.
4) **Ofuscar no es proteger.** Un secreto que el programa necesita para decidir debe quedarse en un servidor, no en el cliente.